Data Structures Lab Fall 2023
Binary Trees

Task One:
Implement a binary tree through array-based implementation. You must make the following classes and functions:

1. You will make a Binary tree class with the following attributes.

```
Class BinaryTree {
  Int * treeArray
  Int capacity
  Int root
  Int current_index
}
```

2. ***BinaryTree():*** You need to make a constructor that allocates memory to the tree array. Then you should initialize the array with -1, denoting that all elements are currently empty. Set your root to 0 and start your current index from 1.

3. ***Void insert():*** Insert the new value into the array at the current index and then increment the current index.

4. ***Void inorderTraversal(int index***) : Function to perform inorder traversal. Note that this function will be called recursively.

```
Base Case: if treearray[index] == -1
Recursive Call left (2*index+1)
Print
Recursive Call right (2*index+2)
```

5. ***Void displayInorder():*** Function that will display the inorder traversal. Just call the recursive function with root.

Task Two:

Write a C++ code to implement the node-based implementation of a binary tree. You must implement the following functions.

1. You will need to make a node class that has the following parameters.

   Class Node {
   Int data;
   Node * left;
   Node * right;
   }

2. Now you need to make a Binary tree class that will contain the root of the tree.

   Class BinaryTree{
   Node * Root;
   }

3. Void insert(int data):

You can insert the node in the following manner if you want to insert the node of the left child.
Following is an example of how you will add a node to left of root node

```
//Step 1: Make a new tree node;
Node * newNode = new Node(data);
//Step 2: Make current node that points to root
Node * currentnode = new Node(data);
//Step 3: Adjust the new node to the left of the current node
current->left = newNode
```

The tricky part in this question is that you have to add nodes in such a way that a complete binary tree is formed.

4. Void InoderTraversal(Node * treeNode):

This is a recursive function that will be called to traverse the tree in order.

5) Void Display()

This function will just call inoderTraversal function with root of tree

Task 3:

You are given a binary tree where each node contains a lowercase letter. Your task is to determine whether a given word can be formed by traversing the tree using Depth-First Search (DFS) with a preorder traversal.

Input:

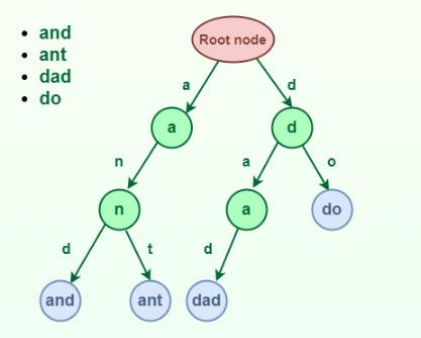
- A binary tree where each node contains a lowercase letter.
- A string representing the word to be searched for.

Output:

- Return `True` if the given word can be formed by traversing the binary tree using DFS (preorder traversal), or `False` if it cannot be formed.

Example:

Tree:



Word = "ANT"

Output: True

The word 'ANT' can be formed in the tree.

HOME TASKS (Non Graded)

- Write a C++ code to make array based tree and write a function that checks if it is complete binary tree
- Write a C++ code to make array based tree and write a function that checks if it is a balanced tree
- Write a C++ code to make array based tree and write a function that checks if it is full binary tree
- Write a C++ code to make array based tree and write a function that checks if it is complete binary tree